LLM LeetCode Benchmark

How well do open-source LLMs solve 10 hard LeetCode problems in C++? Correctness, speed, memory and code quality of every generated solution. Run 2026-11-04 01:58 · Intel(R) Core(TM) i7-10700F CPU @ 2.90GHz · g++ (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0

35/50	5	221	253,785
solutions accepted (all models)	models · 10 problems each	test files	test cases

Problems accepted (out of 10)

Accepted = passes every edge, random and heavy test of the problem.



Highlights

- **Qwen2.5-Coder-7B-Instruct, phi-4, Qwen2.5-Coder-14B-Instruct** accepted the most problems: 9 of 10. Ties are ranked by the share of tests passed (**Qwen2.5-Coder-7B-Instruct**: 96.7%).
- Solved by every model: 32 (Longest Valid Parentheses), 41 (First Missing Positive), 42 (Trapping Rain Water).
- 35 of 50 solutions were accepted. Failures: 3 wrong answer, 1 runtime error, 2 time limit exceeded, 4 compile error, 5 no code.
- **Qwen3-8B** returned no code for 5 problems (4, 10, 23, 25, 30): the answer had no C++ code block (a reasoning model that runs out of tokens while thinking never writes the answer).
- Problem 44 (Wildcard Matching): accepted solutions were ~324x slower than the reference on heavy tests. Reference: Greedy scan that backtracks to the last '*', O(1) memory (O(|s| ·|p|) worst case).
- Problem 37 (Sudoku Solver): accepted solutions were ~202x slower than the reference on heavy tests. Reference: Backtracking with bitmasks, always filling the cell with the fewest candidates first.
- Memory errors / undefined behavior found by the sanitizers: phi-4 on 25, Qwen3-8B (no think) on 4, Qwen3-8B (no think) on 10.

What was tested

Each model got the problem statement and a C++ starter (function signature only) and had to write the function. Its code is compiled and run on hundreds of generated tests per problem, from tiny corner cases up to LeetCode's maximum input sizes and beyond. The answers are compared with a trusted reference solution, the run time is measured, and the code is checked for memory errors and warnings. Pages 2–6 explain every metric and the test suite; then one page per problem.

Model summary

Ranked by problems accepted, then by share of tests passed, then by speed.

#	Model	Accepted	Tests passed	Compiles	Standalone	Speed vs ref	Memory vs ref	Stress passed	Sanitizer	Warnings	Avg LOC
1	Qwen2.5-Coder-7B-Instruct	9/10	202/209 (96.7%)	10/10	7/10	3.81x	1.14x	11/12	0	2	28
2	phi-4	9/10	197/209 (94.3%)	10/10	8/10	4.77x	1.14x	10/12	1	2	28
3	Qwen2.5-Coder-14B-Instruct	9/10	184/209 (88.0%)	9/10	7/10	6.92x	1.16x	9/12	0	0	27
4	Qwen3-8B	5/10	100/209 (47.8%)	5/10	5/10	9.58x	1.19x	4/12	0	1	27
5	Qwen3-8B (no think)	3/10	101/209 (48.3%)	7/10	6/10	1.06x	0.97x	4/12	2	5	31

What each column means

Accepted — Problems where the solution passed **every** edge, random and heavy test file — the same bar as LeetCode's "Accepted". A single wrong case, crash or timeout fails the problem.

Tests passed — Test files passed out of all edge + random + heavy files of the 10 problems (a missing or non-compiling solution counts as 0 passed). Finer-grained than Accepted: it shows how close a failing solution got.

Compiles — Solutions that compile in a LeetCode-like environment: `#include <bits/stdc++.h>` and `using namespace std;` are added before the code, as LeetCode's judge does. Missing code counts as not compiling.

Standalone — Solutions whose untouched file compiles on its own, with only the #includes the model wrote. Measures whether the model followed the prompt ("include every #include the code needs"). Failing this alone does not fail a problem.

Speed vs ref — How much slower than the reference solution: for each heavy test, solution time ÷ reference time; then the geometric mean over the heavy tests of a problem, and over the accepted problems of a model. 1× = as fast, 3× = three times slower, 0.9× = 10% faster. Only accepted problems count, so a model with few (easy) accepted problems can look fast.

Memory vs ref — Same idea for memory: peak memory the function calls added (input excluded), ÷ the reference's. 64 KB is added to both sides so tiny values (a few KB) don't create huge ratios.

Stress passed — Stress tests passed. They go 10× to 1000× beyond LeetCode's limits (e.g. a million list nodes) to show how solutions scale; they **don't** count for Accepted.

Sanitizer — Problems where AddressSanitizer or UndefinedBehaviorSanitizer reported an error (out-of-bounds access, use of invalid values, crash…) on the edge and random tests. Such code can pass by luck and break on another compiler or machine.

Warnings — Compiler warnings (`-Wall -Wextra`) inside the AI code, e.g. comparing signed and unsigned integers.

Avg LOC — Average lines of code per solution, without blank lines and comments.

Verdict per problem

One cell per (model, problem). Accepted cells show the speed vs the reference; failing cells show the verdict and how many of the problem's counted test files passed.

Model	4	10	23	25	30	32	37	41	42	44
Qwen2.5-Coder-7B-Instruct	✓ AC 1.04x	✓ AC 1.59x	✓ AC 1.88x	✓ AC 0.88x	✗ RE 15/22	✓ AC 1.09x	✓ AC 188x	✓ AC 1.00x	✓ AC 0.92x	✓ AC 326x
phi-4	✓ AC 1.03x	✓ AC 1.54x	✓ AC 1.10x	⏱ TLE 5/17	✓ AC 3.45x	✓ AC 1.07x	✓ AC 219x	✓ AC 1.00x	✓ AC 3.08x	✓ AC 292x
Qwen2.5-Coder-14B-Instruct	⚠ CE	✓ AC 1.60x	✓ AC 1.09x	✓ AC 1.16x	✓ AC 87x	✓ AC 1.15x	✓ AC 195x	✓ AC 1.03x	✓ AC 3.00x	✓ AC 301x
Qwen3-8B	∅ No code	∅ No code	∅ No code	∅ No code	∅ No code	✓ AC 1.19x	✓ AC 207x	✓ AC 1.01x	✓ AC 0.85x	✓ AC 383x
Qwen3-8B (no think)	✗ WA 11/25	✗ WA 12/25	⚠ CE	⚠ CE	✗ WA 7/22	✓ AC 1.10x	⚠ CE	✓ AC 1.25x	✓ AC 0.87x	⏱ TLE 10/26

4 = Median of Two Sorted Arrays · 10 = Regular Expression Matching · 23 = Merge k Sorted Lists · 25 = Reverse Nodes in k-Group · 30 = Substring with Concatenation of All Words · 32 = Longest Valid Parentheses · 37 = Sudoku Solver · 41 = First Missing Positive · 42 = Trapping Rain Water · 44 = Wildcard Matching

Verdicts

✓ AC	Accepted: every counted test passed.
✗ WA	Wrong Answer: the program ran, but at least one case gave a different answer than the reference.
✗ RE	Runtime Error: crash (segmentation fault, stack overflow), uncaught exception or abort.
⏱ TLE	Time Limit Exceeded: a test file took more than 10 s (killed). Includes infinite loops.
✗ MLE	Memory Limit Exceeded: allocation failed above 4096 MiB.
⚠ CE	Compile Error: the code does not compile, even with LeetCode's implicit headers.
∅ No code	No code: the model's answer had no C++ code block to extract.

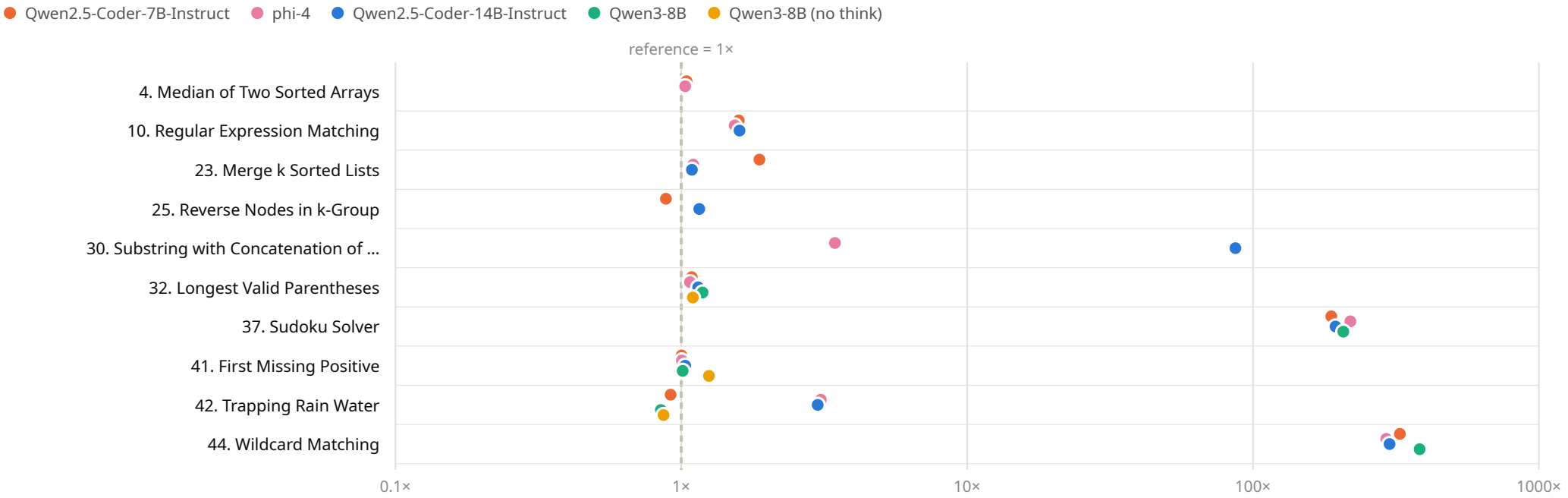
How the verdict is chosen

Like LeetCode, the verdict is the result of the **first failing test**, in the order edge → random → heavy (inside a tier, by file name). So a solution that is wrong on a corner case shows WA even if it would also be too slow on big inputs. Every test still runs, so “Tests passed” counts all of them.

Stress tests are reported separately (column “Stress passed”) and never change the verdict.

Runtime vs reference (heavy tests)

Accepted solutions only. Each dot: geometric mean over the problem's heavy tests of (solution time ÷ reference time). Log scale: every grid line is 10× slower; left of the dashed line = faster than the reference.



Total time of the solution function on all heavy tests of the problem

Problem	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
4. Median of Two Sorted Arrays	0.853 ms	0.899 ms	0.883 ms	—	—	—
10. Regular Expression Matching	187 ms	261 ms	274 ms	264 ms	—	—
23. Merge k Sorted Lists	40.7 ms	56.8 ms	45.8 ms	46.1 ms	—	—
25. Reverse Nodes in k-Group	9.7 ms	8.6 ms	—	11.4 ms	—	—
30. Substring with Concatenation of All Words	5.7 ms	3,929 ms	16.8 ms	3,896 ms	—	—
32. Longest Valid Parentheses	11.5 ms	12.7 ms	12.4 ms	12.7 ms	13.6 ms	13.1 ms
37. Sudoku Solver	26.3 ms	6,892 ms	8,125 ms	7,271 ms	7,805 ms	—
41. First Missing Positive	42.9 ms	43.0 ms	43.1 ms	44.0 ms	43.8 ms	47.7 ms
42. Trapping Rain Water	1.7 ms	1.5 ms	5.4 ms	5.2 ms	1.4 ms	1.5 ms
44. Wildcard Matching	5.4 ms	344 ms	323 ms	316 ms	482 ms	—

— = not all heavy tests passed. Times are the median of 3 runs, measured inside the program around the function calls only.

How the benchmark works

Pipeline

1. **Generation** (`generation/generate.py`): each model receives the LeetCode statement and a C++ starter with the function signature, and must return only the function in a ```` ```cpp ```` block. Greedy decoding (no sampling), so the same prompt gives the same code.
2. **Treatment**: the code is copied to `treated_generated_code/` byte for byte; only LeetCode's implicit preamble and a `main()` that reads the input, calls the function, times it and prints the output are added around it. Wrong code is never fixed.
3. **Build**: `-std=c++20 -O2 -Wall -Wextra`, plus a sanitizer build (`-fsanitize=address,undefined`) and a standalone syntax check of the original file.
4. **Expected answers**: hand-written reference solutions (efficient, well-known algorithms) produce the answers; on every edge and random case they are double-checked against simple brute-force Python oracles. The run stops if they ever disagree.
5. **Runs**: edge/random tests in parallel; heavy/stress tests one at a time, 3 runs each, median time kept; sanitizer build on edge/random tests.
6. **Judging**: outputs compared case by case (any order for problem 30, $\pm 10^{-5}$ for problem 4; sudoku boards must equal the unique solution).

Measurements

- **Time**: `steady_clock` around each call to the solution function, summed over the cases of the file. Reading and parsing the input is excluded.
- **Memory**: the kernel's peak-RSS counter is reset after the input is loaded and read after the calls: the memory the solution itself allocated.
- **Limits**: 10 s per test file (20 s for stress), 8 MiB stack (Linux default), 4096 MiB address space.
- **Machine**: Intel(R) Core(TM) i7-10700F CPU @ 2.90GHz, 16 threads, Linux-7.0.0-38-generic-x86_64-with-glibc2.39.

Test tiers

Tier	What it is	Files	Cases	Counts for Accepted
edge	One hand-written corner case per file: LeetCode's examples, empty inputs, single elements, extreme values, known traps.	127	127	yes
random	Hundreds or thousands of small random cases per file, compared with the oracles. Finds logic errors the edge cases miss.	50	26,700	yes
heavy	LeetCode's maximum input sizes, many cases per file, and adversarial inputs (exponential backtracking traps, the hardest known sudokus). Correctness + timing.	32	226,915	yes
stress	10× to 1000× beyond LeetCode's limits. Shows how a solution scales (e.g. recursion that overflows the stack, $O(n^2)$ that times out). Timing only.	12	43	no

All tests are generated deterministically (seed 20261104), so every run uses exactly the same inputs.

Code-quality checks

- **Sanitizers**: AddressSanitizer (out-of-bounds, use-after-free, stack overflow) and UndefinedBehaviorSanitizer (invalid bool values, signed overflow, ...).
- **Standalone compile**: the original file, with only its own #includes.
- **Warnings**: `-Wall -Wextra`, counted only inside the AI code.
- **Prints**: the prompt forbids printing; output to stdout is flagged.
- **Size**: lines of code without blanks and comments.

The 10 problems and their tests

All are LeetCode **Hard** problems. Constraints are LeetCode's; heavy tests reach them, stress tests exceed them.

#	Problem	LeetCode constraints	Reference algorithm	Edge	Random	Heavy	Stress	Cases
4	**Median of Two Sorted Arrays** Return the median of two sorted arrays (LeetCode asks for O(log(m+n)) time).	$0 \le m, n \le 1000 \cdot 1 \le m + n \le 2000 \cdot -10^6 \le value \le 10^6$	Binary search on the partition of the smaller array, O(log min(m, n)).	17	5	3	1	7,519
10	**Regular Expression Matching** Regular expression matching with '.' (any char) and '*' (zero or more of the previous element), covering the entire string.	$1 \le \lvert s\rvert, \lvert p\rvert \le 20$ · lowercase letters, '.', '*' · every '*' follows a valid character	Bottom-up dynamic programming, O(|s|·|p|).	18	5	2	2	230,049
23	**Merge k Sorted Lists** Merge k sorted linked lists into one sorted linked list.	$0 \le k \le 10^4$ · each list ≤ 500 nodes · total ≤ 10^4 nodes · $-10^4 \le value \le 10^4$	Min-heap of the list heads, O(N log k).	11	5	4	2	1,633
25	**Reverse Nodes in k-Group** Reverse the nodes of a linked list k at a time; a final group shorter than k stays as it is.	$1 \le k \le n \le 5000 \cdot 0 \le value \le 1000$	Iterative in-place reversal, O(n) time, O(1) extra memory.	10	5	2	2	1,912
30	**Substring with Concatenation of All Words** Find every start index of a substring that is a concatenation of all the words, in any order.	$1 \le \lvert s\rvert \le 10^4 \cdot 1 \le$ #words ≤ 5000 · $1 \le \lvert word\rvert \le 30$ · all words have the same length	Sliding window for each offset over precomputed word ids, O(|s|·|word|).	12	5	5	1	1,532
32	**Longest Valid Parentheses** Length of the longest well-formed (valid) parentheses substring.	$0 \le \lvert s\rvert \le 3 \cdot 10^4$ · s contains only '(' and ')'	Stack of indices, O(n).	12	5	2	1	2,619
37	**Sudoku Solver** Fill the empty cells of a 9×9 sudoku (the input has exactly one solution).	9×9 board · digits 1–9 or '.' · unique solution	Backtracking with bitmasks, always filling the cell with the fewest candidates first.	5	5	3	0	227
41	**First Missing Positive** Smallest missing positive integer (LeetCode asks for O(n) time and O(1) extra memory).	$1 \le n \le 10^5 \cdot -2^{31} \le value \le 2^{31} - 1$	Cyclic placement (value v goes to index v − 1), O(n) time, O(1) memory.	14	5	4	1	1,595
42	**Trapping Rain Water** How much rain water is trapped between bars of the given heights.	$1 \le n \le 2 \cdot 10^4 \cdot 0 \le height \le 10^5$	Two pointers, O(n) time, O(1) memory.	12	5	2	1	1,617
44	**Wildcard Matching** Wildcard matching with '?' (any single char) and '*' (any sequence), covering the entire string.	$0 \le \lvert s\rvert, \lvert p\rvert \le 2000$ · s lowercase · p lowercase, '?', '*'	Greedy scan that backtracks to the last '*', O(1) memory (O(|s|·|p|) worst case).	16	5	5	1	5,082

4. Median of Two Sorted Arrays · 2/5 models accepted

Return the median of two sorted arrays (LeetCode asks for O(log(m+n)) time).
constraints 0 ≤ m, n ≤ 1000 · 1 ≤ m + n ≤ 2000 · $-10^6 \le$ value $\le 10^6$
reference Binary search on the partition of the smaller array, O(log min(m, n)).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	25/25	1/1	1.04x	31
phi-4	✓ Accepted	25/25	1/1	1.03x	34
Qwen2.5-Coder-14B-Instruct	⚠ Compile Error	0/25	0/1	—	36
Qwen3-8B	∅ No code	0/25	0/1	—	—
Qwen3-8B (no think)	✗ Wrong Answer	11/25	0/1	—	32

Issues found

- **Qwen2.5-Coder-7B-Instruct**
  - not standalone: `4_prompt0.cpp:18:40: error: ‘INT_MIN’ was not declared in this scope`
- **Qwen2.5-Coder-14B-Instruct**
  - compile error: `4_prompt0.cpp:21:16: error: ‘findKthElement’ was not declared in this scope`
- **Qwen3-8B**
  - no code generated
- **Qwen3-8B (no think)**
  - not standalone: `4_prompt0.cpp:15:16: error: ‘INT_MIN’ was not declared in this scope`
  - first failure **edge_01** (example 1): Wrong Answer — case 1: expected 2.00000, got 3.00000
  - sanitizer (edge_02): `SUMMARY: AddressSanitizer: heap-buffer-overflow 4_prompt0.cpp:38 in findMedianSortedArrays(std::vector<in…`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; second array empty; first array empty; first empty, even length; equal single elements; all duplicates; only first array, odd; first array longer; single element in the middle; single element smaller than all; min values; max values; extreme values, even; disjoint ranges; disjoint ranges, reversed; negatives interleaved	17	17/17	17/17 ✓	17/17 ✓	—	—	11/17 ✗ WA×5 RE×1
`random_*`	5 files of small random cases (300 random cases, sizes 0..40 each)	1,500	5/5	5/5 ✓	5/5 ✓	—	—	0/5 ✗ RE×5
`heavy_01`	2000 cases, m = n = 1000, random	2,000	0.361 ms	0.403 ms	0.376 ms	—	—	✗ WA
`heavy_02`	2000 cases, m = n = 1000, disjoint ranges	2,000	0.284 ms	0.283 ms	0.299 ms	—	—	✗ WA
`heavy_03`	2000 cases, unbalanced sizes	2,000	0.209 ms	0.213 ms	0.209 ms	—	—	✗ RE
`stress_01`	m = n = 1e6 (1000x the constraint)	2	0.001 ms	0.001 ms	0.001 ms	—	—	✗ WA

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

10. Regular Expression Matching · 3/5 models accepted

Regular expression matching with '.' (any char) and '*' (zero or more of the previous element), covering the entire string.
constraints 1 ≤ |s|, |p| ≤ 20 · lowercase letters, '.', '*' · every '*' follows a valid character
reference Bottom-up dynamic programming, O(|s|·|p|).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	25/25	2/2	1.59x	25
phi-4	✓ Accepted	25/25	2/2	1.54x	18
Qwen2.5-Coder-14B-Instruct	✓ Accepted	25/25	2/2	1.60x	25
Qwen3-8B	∅ No code	0/25	0/2	—	—
Qwen3-8B (no think)	✗ Wrong Answer	12/25	1/2	—	22

Issues found

- **Qwen2.5-Coder-7B-Instruct**
  - not standalone: `10_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **phi-4**
  - not standalone: `10_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **Qwen2.5-Coder-14B-Instruct**
  - not standalone: `10_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **Qwen3-8B**
  - no code generated
- **Qwen3-8B (no think)**
  - first failure **edge_01** (example 1): Wrong Answer — case 1: expected false, got true
  - sanitizer (edge_01): `runtime error: load of value 2, which is not a valid value for type 'bool'`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; example 3; zero repetitions; classic false; classic true; trailing star zero; dot-star then missing char; star then literal; many optional groups; pattern needs more chars; backtrack into dot-star; dot-star then dots; single dot; 20 dots; exponential backtracking trap (false); many stars (true); all dot-stars	18	18/18	18/18 ✓	18/18 ✓	18/18 ✓	—	11/18 ✗ WA×7
`random_*`	5 files of small random cases (2000 random cases, alphabet 'ab', lengths <= 20 each)	10,000	5/5	5/5 ✓	5/5 ✓	5/5 ✓	—	0/5 ✗ WA×5
`heavy_01`	200k random cases at max length 20	200,000	170 ms	229 ms	246 ms	232 ms	—	✗ WA
`heavy_02`	20k backtracking traps like s=a..ab, p=a*a*..a*c	20,000	16.9 ms	31.6 ms	27.6 ms	31.6 ms	—	8.0 ms
`stress_01`	30 cases, |s|, |p| ~ 1000 (50x the constraint)	30	28.9 ms	33.9 ms	37.4 ms	33.2 ms	—	✗ WA
`stress_02`	trap with |s| = 1001, 300 stars	1	0.940 ms	1.7 ms	1.2 ms	1.7 ms	—	0.564 ms

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

23. Merge k Sorted Lists · 3/5 models accepted

Merge k sorted linked lists into one sorted linked list.

constraints $0 \le k \le 10^4$ · each list ≤ 500 nodes · total ≤ 10^4 nodes · $-10^4 \le$ value $\le 10^4$

reference Min-heap of the list heads, O(N log k).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	20/20	2/2	1.88x	36
phi-4	✓ Accepted	20/20	2/2	1.10x	35
Qwen2.5-Coder-14B-Instruct	✓ Accepted	20/20	2/2	1.09x	35
Qwen3-8B	∅ No code	0/20	0/2	—	—
Qwen3-8B (no think)	⚠ Compile Error	0/20	0/2	—	34

Issues found

- **Qwen3-8B**
  - no code generated
- **Qwen3-8B (no think)**
  - compile error: `23_prompt0.cpp:15:10: fatal error: priority_queue: No such file or directory`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; k = 0; one empty list; two empty lists; one single-node list; empty then non-empty; two single nodes, reversed; all equal values; extreme values; one long list among empties; 100 single-node lists, descending heads	11	11/11	11/11 ✓	11/11 ✓	11/11 ✓	—	—
`random_*`	5 files of small random cases (300 random cases, k <= 20, lengths <= 20 each)	1,500	5/5	5/5 ✓	5/5 ✓	5/5 ✓	—	—
`heavy_01`	30 cases, k = 10000 lists of 1 node	30	23.1 ms	22.2 ms	25.9 ms	26.7 ms	—	—
`heavy_02`	30 cases, k = 20 lists of up to 500 nodes	30	3.4 ms	6.4 ms	3.7 ms	3.8 ms	—	—
`heavy_03`	30 cases, k = 100 lists of 100 nodes	30	11.9 ms	17.9 ms	13.8 ms	13.4 ms	—	—
`heavy_04`	30 cases, 20 lists of 500 equal values	30	2.2 ms	10.4 ms	2.3 ms	2.2 ms	—	—
`stress_01`	k = 10000 lists of 100 nodes (1e6 nodes, 100x the constraint)	1	154 ms	406 ms	126 ms	135 ms	—	—
`stress_02`	k = 100000 lists of 5 nodes	1	127 ms	177 ms	90.7 ms	97.5 ms	—	—

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

25. Reverse Nodes in k-Group · 2/5 models accepted

Reverse the nodes of a linked list k at a time; a final group shorter than k stays as it is.

constraints 1 ≤ k ≤ n ≤ 5000 · 0 ≤ value ≤ 1000

reference Iterative in-place reversal, O(n) time, O(1) extra memory.

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	17/17	2/2	0.88x	36
phi-4	⏱ Time Limit Exceeded	5/17	0/2	—	36
Qwen2.5-Coder-14B-Instruct	✓ Accepted	17/17	1/2	1.16x	29
Qwen3-8B	∅ No code	0/17	0/2	—	—
Qwen3-8B (no think)	⚠ Compile Error	0/17	0/2	—	44

Issues found

- **phi-4**
  - first failure **edge_01** (example 1): Time Limit Exceeded
  - sanitizer (edge_02): `SUMMARY: AddressSanitizer: SEGV 25_prompt0.cpp:53 in reverseKGroup(ListNode*, int)`
- **Qwen3-8B**
  - no code generated
- **Qwen3-8B (no think)**
  - compile error: `25_prompt0.cpp:28:55: error: a function-definition is not allowed here before '{' token`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; k = 1; k = n; k = n - 1; single node; two nodes, k = 2; odd length, k = 2; exact multiple of k; boundary values	10	10/10	10/10 ✓	5/10 ⏱ TLE×3 RE×2	10/10 ✓	—	—
`random_*`	5 files of small random cases (300 random cases, n <= 50 each)	1,500	5/5	5/5 ✓	0/5 ⏱ TLE×4 RE×1	5/5 ✓	—	—
`heavy_01`	200 cases, n = 5000, k in {1,2,3,7,50,2500,4999,5000}	200	5.2 ms	4.3 ms	⏱ TLE	4.7 ms	—	—
`heavy_02`	200 cases, n = 5000, k = 2	200	4.6 ms	4.3 ms	⏱ TLE	6.7 ms	—	—
`stress_01`	n = 1e6, k = 2 (200x the constraint)	1	3.9 ms	2.7 ms	⏱ TLE	✗ RE	—	—
`stress_02`	n = 1e6, k = 1000	1	4.2 ms	4.6 ms	⏱ TLE	3.7 ms	—	—

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

30. Substring with Concatenation of All Words · 2/5 models accepted

Find every start index of a substring that is a concatenation of all the words, in any order.
constraints 1 ≤ |s| ≤ 10^4 · 1 ≤ #words ≤ 5000 · 1 ≤ |word| ≤ 30 · all words have the same length
reference Sliding window for each offset over precomputed word ids, O(|s|·|word|).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✗ Runtime Error	15/22	0/1	—	29
phi-4	✓ Accepted	22/22	1/1	3.45x	42
Qwen2.5-Coder-14B-Instruct	✓ Accepted	22/22	0/1	87x	27
Qwen3-8B	∅ No code	0/22	0/1	—	—
Qwen3-8B (no think)	✗ Wrong Answer	7/22	0/1	—	46

Issues found

- **Qwen2.5-Coder-7B-Instruct**
  - first failure **edge_08** (word longer than s): Runtime Error — exception std::out_of_range: basic_string::substr: __pos (which is 3) > this->size() (which is 2)
  - warning: `line 19: comparison of integer expressions of different signedness: ‘int’ and ‘std::__cxx11::basic_string…`
- **phi-4**
  - warning: `line 13: comparison of integer expressions of different signedness: ‘std::__cxx11::basic_string<char>::si…`
- **Qwen3-8B**
  - no code generated
- **Qwen3-8B (no think)**
  - first failure **edge_01** (example 1): Wrong Answer — case 1: expected 0 9, got
  - warning: `line 23: comparison of integer expressions of different signedness: ‘int’ and ‘std::__cxx11::basic_string…`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; example 3; duplicate words; overlapping matches; single char match; single char no match; word longer than s; words longer than s in total; non-aligned offsets; classic tricky case; word equals s	12	12/12	10/12 ✗ RE×2	12/12 ✓	12/12 ✓	—	5/12 ✗ WA×7
`random_*`	5 files of small random cases (300 random cases, |s| <= 40 each)	1,500	5/5	0/5 ✗ RE×5	5/5 ✓	5/5 ✓	—	0/5 ✗ WA×5
`heavy_01`	s = a^10000, 5000 words 'a' (5001 matches)	1	0.149 ms	1,055 ms	0.743 ms	1,055 ms	—	✗ WA
`heavy_02`	3 cases, s = a^10000, 333 words of length 30	3	0.687 ms	0.799 ms	2.2 ms	0.816 ms	—	✗ WA
`heavy_03`	5 cases, random s over 'ab', 500 words of length 2	5	1.3 ms	1,322 ms	6.2 ms	1,328 ms	—	✗ WA
`heavy_04`	5 cases, s = (ab)^5000, 2500 words	5	0.795 ms	1,551 ms	4.6 ms	1,512 ms	—	4.0 ms
`heavy_05`	5 cases, random lowercase s, 2000 words of length 5	5	2.8 ms	0.755 ms	3.1 ms	0.840 ms	—	3.2 ms
`stress_01`	s = a^100000, 50000 words 'a' (10x the constraints)	1	1.6 ms	⏱ TLE	7.3 ms	⏱ TLE	—	✗ WA

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

32. Longest Valid Parentheses · 5/5 models accepted

Length of the longest well-formed (valid) parentheses substring.

constraints $0 \le |s| \le 3 \cdot 10^4$ · s contains only '(' and ')'

reference Stack of indices, O(n).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	19/19	1/1	1.09x	24
phi-4	✓ Accepted	19/19	1/1	1.07x	20
Qwen2.5-Coder-14B-Instruct	✓ Accepted	19/19	1/1	1.15x	19
Qwen3-8B	✓ Accepted	19/19	1/1	1.19x	23
Qwen3-8B (no think)	✓ Accepted	19/19	1/1	1.10x	25

Issues found

- **Qwen2.5-Coder-7B-Instruct**
  - warning: `line 8: comparison of integer expressions of different signedness: ‘int’ and ‘std::__cxx11::basic_string<…`
- **Qwen3-8B**
  - warning: `line 10: comparison of integer expressions of different signedness: ‘int’ and ‘std::__cxx11::basic_string…`
- **Qwen3-8B (no think)**
  - warning: `line 10: comparison of integer expressions of different signedness: ‘int’ and ‘std::__cxx11::basic_string…`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; empty string; single open; single close; single pair; valid then unclosed; nested; no valid pair; concatenated; mostly open; long valid in the middle	12	12/12	12/12 ✓	12/12 ✓	12/12 ✓	12/12 ✓	12/12 ✓
`random_*`	5 files of small random cases (500 random cases, `|s| <= 60` each)	2,500	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓
`heavy_01`	100 cases, $\|s\| = 30000$, random	100	11.3 ms	12.5 ms	12.2 ms	12.4 ms	13.3 ms	12.8 ms
`heavy_02`	5 structured cases at $\|s\| = 30000$	5	0.210 ms	0.224 ms	0.224 ms	0.249 ms	0.250 ms	0.222 ms
`stress_01`	$\|s\| = 1e7$ (333x the constraint)	2	64.3 ms	66.1 ms	81.4 ms	84.3 ms	67.4 ms	65.7 ms

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

37. Sudoku Solver · 4/5 models accepted

Fill the empty cells of a 9×9 sudoku (the input has exactly one solution).

constraints 9×9 board · digits 1–9 or '.' · unique solution

reference Backtracking with bitmasks, always filling the cell with the fewest candidates first.

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	13/13	0/0	188x	36
phi-4	✓ Accepted	13/13	0/0	219x	30
Qwen2.5-Coder-14B-Instruct	✓ Accepted	13/13	0/0	195x	41
Qwen3-8B	✓ Accepted	13/13	0/0	207x	42
Qwen3-8B (no think)	⚠ Compile Error	0/13	0/0	—	45

Issues found

- **Qwen3-8B (no think)**
  - compile error: `37_prompt0.cpp:31:42: error: no match for ‘operator=’ (operand types are ‘__gnu_cxx::__alloc_traits<std::…`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: LeetCode example; only the first cell empty; only the last cell empty; first row empty; middle row empty	5	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	—
`random_*`	5 files of small random cases (40 random unique puzzles with 28..40 clues each)	200	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	—
`heavy_01`	15 minimal random puzzles (no clue can be removed)	15	0.698 ms	96.7 ms	111 ms	98.0 ms	104 ms	—
`heavy_02`	known hardest puzzles: Arto Inkala 2012, Easter Monster, Norvig hard 1, Norvig hard 2, Norvig hard 3, Norvig hard 4	6	18.0 ms	1,160 ms	1,355 ms	1,194 ms	1,261 ms	—
`heavy_03`	puzzle built to defeat row-major brute force (first row is 987654321)	1	7.6 ms	5,635 ms	6,658 ms	5,978 ms	6,439 ms	—

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

41. First Missing Positive · 5/5 models accepted

Smallest missing positive integer (LeetCode asks for O(n) time and O(1) extra memory).

constraints $1 \le n \le 10^5 \cdot -2^{31} \le \text{value} \le 2^{31} - 1$

reference Cyclic placement (value v goes to index v − 1), O(n) time, O(1) memory.

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	23/23	1/1	1.00x	16
phi-4	✓ Accepted	23/23	1/1	1.00x	16
Qwen2.5-Coder-14B-Instruct	✓ Accepted	23/23	1/1	1.03x	17
Qwen3-8B	✓ Accepted	23/23	1/1	1.01x	17
Qwen3-8B (no think)	✓ Accepted	23/23	1/1	1.25x	15

Issues found

- **Qwen3-8B (no think)**
  - warning: `line 5: comparison of integer expressions of different signedness: 'int' and 'std::vector<int>::size_type…`

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; example 3; single 1; single 2; single negative; single zero; duplicate ones; duplicates, answer 1; int32 extremes; only INT_MAX; shuffled permutation 1..100 (answer n+1); reversed permutation; gap after 3	14	14/14	14/14 ✓	14/14 ✓	14/14 ✓	14/14 ✓	14/14 ✓
`random_*`	5 files of small random cases (300 random cases, n <= 50 each)	1,500	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓
`heavy_01`	20 cases, shuffled permutation of 1..1e5	20	18.0 ms	18.3 ms	18.5 ms	18.5 ms	17.7 ms	19.6 ms
`heavy_02`	20 cases, permutation with one value removed	20	17.3 ms	17.1 ms	17.0 ms	17.9 ms	18.5 ms	19.1 ms
`heavy_03`	20 cases, n = 1e5, full int32 range	20	6.2 ms	6.2 ms	6.2 ms	6.1 ms	6.2 ms	6.2 ms
`heavy_04`	20 cases, n = 1e5, heavy duplicates	20	1.4 ms	1.4 ms	1.4 ms	1.5 ms	1.4 ms	2.8 ms
`stress_01`	shuffled permutation, n = 3e6 (30x)	1	51.2 ms	57.9 ms	54.9 ms	57.0 ms	56.9 ms	61.0 ms

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

42. Trapping Rain Water · 5/5 models accepted

How much rain water is trapped between bars of the given heights.

constraints $1 \le n \le 2 \cdot 10^4 \cdot 0 \le height \le 10^5$

reference Two pointers, O(n) time, O(1) memory.

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	19/19	1/1	0.92x	25
phi-4	✓ Accepted	19/19	1/1	3.08x	22
Qwen2.5-Coder-14B-Instruct	✓ Accepted	19/19	1/1	3.00x	21
Qwen3-8B	✓ Accepted	19/19	1/1	0.85x	25
Qwen3-8B (no think)	✓ Accepted	19/19	1/1	0.87x	27

Issues found

None.

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; single bar; two bars; simple valley; all zeros; increasing; decreasing; small pocket; max height valley; flat; nested pockets	12	12/12	12/12 ✓	12/12 ✓	12/12 ✓	12/12 ✓	12/12 ✓
`random_*`	5 files of small random cases (300 random cases, n <= 60 each)	1,500	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓
`heavy_01`	100 cases, n = 20000, random heights	100	1.6 ms	1.5 ms	5.2 ms	5.0 ms	1.4 ms	1.4 ms
`heavy_02`	structured: huge valley (answer ~2e9), mountain, increasing, sawtooth	4	0.074 ms	0.069 ms	0.221 ms	0.216 ms	0.064 ms	0.064 ms
`stress_01`	n = 2e6 (100x the constraint), heights <= 500	1	1.4 ms	1.4 ms	8.2 ms	8.8 ms	1.2 ms	1.4 ms

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

44. Wildcard Matching · 4/5 models accepted

Wildcard matching with '?' (any single char) and '*' (any sequence), covering the entire string.

constraints 0 ≤ |s|, |p| ≤ 2000 · s lowercase · p lowercase, '?', '*'

reference Greedy scan that backtracks to the last '*', O(1) memory (O(|s|·|p|) worst case).

Results

Model	Verdict	Tests	Stress	Speed	LOC
Qwen2.5-Coder-7B-Instruct	✓ Accepted	26/26	1/1	326x	22
phi-4	✓ Accepted	26/26	1/1	292x	22
Qwen2.5-Coder-14B-Instruct	✓ Accepted	26/26	1/1	301x	22
Qwen3-8B	✓ Accepted	26/26	1/1	383x	26
Qwen3-8B (no think)	⏱ Time Limit Exceeded	10/26	0/1	—	22

Issues found

- **Qwen2.5-Coder-7B-Instruct**
  - not standalone: `44_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **phi-4**
  - not standalone: `44_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **Qwen2.5-Coder-14B-Instruct**
  - not standalone: `44_prompt0.cpp:6:12: error: ‘vector’ was not declared in this scope`
- **Qwen3-8B (no think)**
  - first failure **edge_02** (example 2): Time Limit Exceeded

Tests

Test	Description	Cases	Reference	Qwen2.5-Coder-7B-Instruct	phi-4	Qwen2.5-Coder-14B-Instruct	Qwen3-8B	Qwen3-8B (no think)
`edge_*`	one corner case per file: example 1; example 2; example 3; stars around letters; classic false; both empty; empty s, star; empty s, many stars; empty s, question mark; empty pattern; consecutive stars; classic mixed false; stars then letter; five ? between stars; one ? too many; backtracking trap	16	16/16	16/16 ✓	16/16 ✓	16/16 ✓	16/16 ✓	10/16 ⏱ TLE×6
`random_*`	5 files of small random cases (1000 random cases, lengths <= 25 each)	5,000	5/5	5/5 ✓	5/5 ✓	5/5 ✓	5/5 ✓	0/5 ⏱ TLE×5
`heavy_01`	5 cases, s = a^2000, p = *a*a...*b (exponential for naive recursion)	5	0.028 ms	41.5 ms	31.1 ms	37.2 ms	36.6 ms	⏱ TLE
`heavy_02`	30 random cases, lengths up to 2000	30	0.169 ms	63.6 ms	68.3 ms	59.1 ms	103 ms	⏱ TLE
`heavy_03`	20 cases, |s| = 2000 with matching-ish patterns	20	0.200 ms	184 ms	176 ms	168 ms	287 ms	⏱ TLE
`heavy_04`	5 cases, s = (ab)^1000, long ?b* chain that fails at the end	5	0.020 ms	33.0 ms	29.2 ms	31.8 ms	34.5 ms	⏱ TLE
`heavy_05`	5 cases, s = a^2000, p = *a^999b (worst case for greedy star backtracking)	5	5.0 ms	21.9 ms	18.5 ms	19.8 ms	20.9 ms	⏱ TLE
`stress_01`	|s| = |p| = 20000 (10x the constraint), trap	1	0.055 ms	809 ms	599 ms	730 ms	713 ms	⏱ TLE

Edge/random rows: files passed (and the failure kinds). Heavy/stress rows: time of the solution function (median of the runs) or the verdict of that file. — = no compiled code. Full per-file data: `benchmark/results/tests.csv`.

Reproducing

From the repository root:

- `.venv/bin/python generation/generate.py` — generates the code (needs a GPU and the Hugging Face models; already done, results in `generated_code/`).
- `.venv/bin/python benchmark/run_benchmark.py` — tests, builds, runs and writes this report (≈10 min on the machine above).
- `.venv/bin/python benchmark/make_report.py` — rebuilds the report from `benchmark/results/results.json` without re-running.

Raw data: `benchmark/results/results.json` (everything), `tests.csv` (one row per test run), `summary.md` (the tables above in Markdown).

Timings depend on the machine; ratios vs the reference are more portable than absolute times.